Worked example: under-5 mortality in Greater Manchester across changing LSOA boundaries

Contents

1 Overview **2**
- 1.1 What DAST does . . . 2
- 1.2 The example . . . 2

2 Data sources, download, and freezing **2**
- 2.1 Deaths . . . 3
- 2.2 Boundaries and lookup . . . 4
- 2.3 Population . . . 5

3 Data preparation **5**
- 3.1 Reconstructing the 2001 geography for 2008–09 . . . 5
- 3.2 One `sf` object per period . . . 6

4 Pre-fit checks **7**
- 4.1 The boundary change . . . 7

5 Model specification **9**
- 5.1 Mesh . . . 9
- 5.2 Priors . . . 9
- 5.3 Likelihood and intercepts . . . 11

6 Fitting **11**

7 Outputs **12**
- 7.1 Parameter summary . . . 12
- 7.2 Relative-risk and exceedance maps . . . 13
- 7.3 Prior and posterior distributions . . . 14

8 Interpretation **15**

9 Common issues and debugging **15**
- 9.1 LSOA vintages and lookups . . . 15
- 9.2 Agencies re-code old years to new boundaries . . . 16
- 9.3 Raster CRS and alignment . . . 16
- 9.4 s2 versus GEOS topology errors . . . 16
- 9.5 AGHQ fails with `eigen()`: "infinite or missing values in 'x'" . . . 16
- 9.6 Other checks worth running . . . 16

10 Applying the method to your own data **17**

11 Using an AI assistant **17**

This document is a worked example of the methods in the paper, implemented in the **DAST** R package. It follows one analysis from raw downloads to interpreted maps, with the checks we recommend at each step. It

can function as a template that can be adapted to other data.

1 Overview

1.1 What DAST does

DAST estimates a continuous risk surface from counts reported for areas (polygons) when the areas change over time. For period i and area j the observed count is modelled as

$$Y_{ij} \sim \text{NegBin}\Big(\sum_{\ell} \mathcal{O}_{ij\ell}\, \exp\{\beta_i + u(q_\ell)\},\ \tau\Big),$$

where $\mathcal{O}_{ij\ell}$ is the population (offset) of grid cell ℓ lying inside area j, β_i is a period-specific intercept, $u(s)$ is a Gaussian random field shared across periods, and τ is the overdispersion parameter. Because the model works at the level of grid cells, each period can have its own set of polygons: 2008–09 counts on one geography and 2010–12 counts on another inform the same surface $u(s)$, and no counts need to be re-allocated between boundary sets beforehand.

1.2 The example

We estimate under-5 mortality risk in Greater Manchester (the ten metropolitan boroughs, E08000001–E08000010) for mid-years 2008–2012, using death counts by Lower Layer Super Output Area (LSOA) from the Office for National Statistics (ONS) and under-5 population from WorldPop. LSOA boundaries were revised after the 2011 Census, so counts for 2008–09 were originally published on 2001 LSOAs and counts for 2010 onwards on 2011 LSOAs.

```
library(dplyr)
library(sf)
library(terra)
library(mapmisc)
library(DAST)    # install.packages("DAST")
library(aghq)
library(ggplot2)

# Use GEOS rather than s2 for polygon operations (see Section 9.4).
sf::sf_use_s2(FALSE)
```

2 Data sources, download, and freezing

Input	Source	Notes
Deaths by LSOA, mid-year 2001–2019	ONS ad hoc release 12626	5-year age bands; *all years on 2011 LSOAs*
LSOA 2001 and 2011 boundaries	ONS Open Geography Portal	generalised, clipped (BGC)
LSOA 2001 to LSOA 2011 lookup	ONS Open Geography Portal	includes change indicator `CHGIND`
Population by age and sex, 100 m	WorldPop Global 2000–2020	age bands 0 (<1) and 1 (1–4)

```
years    <- 2008:2012
gm_lads  <- sprintf("E080000%02d", 1:10)  # Greater Manchester boroughs
uk_grid  <- "EPSG:27700"                  # British National Grid, metres
data_dir <- file.path("data", "manchester")
```

```r
dir.create(data_dir, recursive = TRUE, showWarnings = FALSE)

url_deaths <- paste0(
  "https://www.ons.gov.uk/file?uri=/peoplepopulationandcommunity/",
  "birthsdeathsandmarriages/deaths/adhocs/",
  "12626deathsbylowerlayersuperoutputarealsoaenglandandwalesmidyear2001to2019/",
  "deathsbylsoamidyear01to19.zip")
url_lsoa11 <- paste0("https://opendata.arcgis.com/api/v3/datasets/",
  "02e8d336d6804fbeabe6c972e5a27b16_0/downloads/data?format=geojson",
  "&spatialRefId=4326")
url_lsoa01 <- paste0("https://opendata.arcgis.com/api/v3/datasets/",
  "98b573a6f3ea483e97f7e08771624316_0/downloads/data?format=geojson",
  "&spatialRefId=4326")
url_lookup <- paste0("https://services1.arcgis.com/ESMARspQHYMw9BZ9/ArcGIS/",
  "rest/services/LSOA01_LSOA11_LAD11_EW_LU_ddfe1cd1c2784c9b991cded95bc915a9/",
  "FeatureServer/0/query")
url_worldpop <- "https://data.worldpop.org/GIS/AgeSex_structures/Global_2000_2020"
```

Statistical agencies revise published tables (Section 9.2), and portal URLs change. We therefore **freeze** every processed input the first time it is built: `freeze()` saves the object as a date-stamped `.rds` file and, on later runs, loads the most recent saved copy instead of downloading again. To refresh the data, delete the `.rds` files. The date stamp records when the data were extracted, which belongs in any write-up.

```r
source(file.path("R", "download_helpers.R"))  # download_if_missing() etc.

freeze <- function(name, expr) {
  saved <- sort(list.files(data_dir, paste0("^", name, "_[0-9-]+\\.rds$"),
                           full.names = TRUE), decreasing = TRUE)
  if (length(saved) > 0) {
    message("Loading frozen ", basename(saved[1]))
    out <- readRDS(saved[1])
  } else {
    out <- expr                                  # evaluated only if not frozen
    file <- file.path(data_dir, paste0(name, "_", Sys.Date(), ".rds"))
    saveRDS(if (inherits(out, "SpatRaster")) terra::wrap(out) else out, file)
  }
  if (inherits(out, "PackedSpatRaster")) out <- terra::unwrap(out)
  out
}
```

The inputs used for this document were frozen on 2026-10-26, 2026-10-27.

2.1 Deaths

The ONS workbook has one row per mid-year and LSOA, with a column per age band. "Mid-year 2008" covers deaths from 1 July 2007 to 30 June 2008. We keep the two bands that make up under-5 mortality, `<1` and `01-04`, and name them explicitly rather than matching column names by pattern, so that the age range is visible in the code.

```r
deaths11 <- freeze("deaths_u5_lsoa11", {
  zip <- download_if_missing(url_deaths,
                             file.path(data_dir, "deathsbylsoamidyear01to19.zip"))
  xlsx  <- unzip(zip, exdir = data_dir)
  sheet <- "Deaths - Persons"
  probe <- readxl::read_excel(xlsx, sheet, col_names = FALSE, n_max = 30,
```

```
                                .name_repair = "minimal")
  header_row <- which(tolower(probe[[1]]) == "mid-year")
  raw <- readxl::read_excel(xlsx, sheet, skip = header_row - 1)
  names(raw)[1:4] <- c("year", "lad", "lad_name", "lsoa11cd")
  under5_bands <- c("<1", "01-04")
  stopifnot(all(under5_bands %in% names(raw)))
  # The year is printed only on the first row of each block: fill it down.
  raw$year <- as.integer(sub(".*(20[0-9]{2}).*", "\\1", raw$year))
  raw$year <- data.table::nafill(raw$year, type = "locf")
  out <- raw %>%
    filter(year %in% years, lad %in% gm_lads) %>%
    transmute(year, lad, lad_name, lsoa11cd,
              deaths = as.numeric(`<1`) + as.numeric(`01-04`))
  stopifnot(setequal(unique(out$year), years))  # never freeze an empty table
  out
})
deaths11 %>% group_by(year) %>%
  summarise(lsoas = n(), deaths = sum(deaths), .groups = "drop")
```

```
#> # A tibble: 5 x 3
#>    year lsoas deaths
#>   <int> <int>  <dbl>
#> 1  2008  1673    221
#> 2  2009  1673    228
#> 3  2010  1673    220
#> 4  2011  1673    213
#> 5  2012  1673    202
```

2.2 Boundaries and lookup

```
lookup <- freeze("lsoa01_lsoa11_lookup", {
  fetch_arcgis_attrs(url_lookup,
                     out_fields = c("LSOA01CD", "LSOA11CD", "LAD11CD", "CHGIND"))
}) %>% as.data.frame() %>% filter(LAD11CD %in% gm_lads)

read_lsoa <- function(url, file, code) {
  download_if_missing(url, file.path(data_dir, file))
  st_read(file.path(data_dir, file), quiet = TRUE) %>%
    filter(.data[[code]] %in% unique(lookup[[code]])) %>%
    select(all_of(code)) %>%
    st_transform(uk_grid) %>%
    st_make_valid()
}
lsoa11 <- freeze("lsoa11_gm", read_lsoa(url_lsoa11, "lsoa11_bgc_ew.geojson",
                                        "LSOA11CD"))
lsoa01 <- freeze("lsoa01_gm", read_lsoa(url_lsoa01, "lsoa01_bgc_ew.geojson",
                                        "LSOA01CD"))
c(lsoa01 = nrow(lsoa01), lsoa11 = nrow(lsoa11))
```

```
#> lsoa01 lsoa11
#>   1646   1673
```

```
table(change = lookup$CHGIND)
```

```
#> change
#>    M    S    U    X
#>   12   54 1609    4
```

`CHGIND` describes each 2001–2011 link: `U` unchanged, `S` a 2001 LSOA split into several 2011 LSOAs, `M` several 2001 LSOAs merged into one, and `X` a more complex redrawing. About 4% of Greater Manchester's LSOAs changed.

2.3 Population

WorldPop provides 100 m rasters of population counts by sex and age band. We sum the four under-5 layers (female and male, ages 0 and 1–4) for each year, crop to the study area, and reproject to a 200 m grid on the British National Grid. The reprojection uses `method = "sum"` so that the population totals are preserved (Section 9.3).

```
outline  <- st_union(lsoa11)
template <- rast(ext(vect(st_buffer(outline, 2000))), resolution = 200,
                 crs = uk_grid)

pop_u5 <- freeze("worldpop_u5_gm_200m", {
  layers <- lapply(years, function(yr) {
    files <- sapply(c("f_0", "f_1", "m_0", "m_1"), function(sa) {
      file <- sprintf("worldpop_gbr_%s_%d.tif", sa, yr)
      url  <- sprintf("%s/%d/GBR/gbr_%s_%d.tif", url_worldpop, yr, sa, yr)
      download_if_missing(url, file.path(data_dir, file))
    })
    r <- rast(files)
    r <- crop(r, project(vect(st_buffer(outline, 5000)), crs(r)))
    r <- project(sum(r, na.rm = TRUE), template, method = "sum")
    mask(r, vect(outline))
  })
  out <- rast(layers)
  names(out) <- years
  out
})
round(global(pop_u5, "sum", na.rm = TRUE))
```

```
#>         sum
#> 2008 168690
#> 2009 174167
#> 2010 178421
#> 2011 181433
#> 2012 184263
```

3 Data preparation

DAST needs two named lists with one element per period:

- `poly_list`: an `sf` object of polygons with an ID column and a count column, in a projected CRS;
- `offset_list`: a `SpatRaster` of population counts on a fine grid, in the same CRS.

3.1 Reconstructing the 2001 geography for 2008–09

ONS now distributes these deaths on 2011 LSOAs for *all* years: the 2008–09 counts have been re-coded to the newer geography. Data as they were reported at the time, and as many practitioners will hold them, are

on 2001 LSOAs. To reproduce that situation, we aggregate the 2008–09 counts back to the 2001 geography.

For unchanged and split LSOAs this is exact, because the 2011 children of a split 2001 LSOA add up to the 2001 parent. Where 2001 LSOAs were merged or redrawn, the 2001 parts cannot be recovered from 2011-coded counts, so we dissolve the affected 2001 polygons into a single unit. The result is the finest 2001-based geography recoverable from the published counts. In graph terms, each unit is a connected component of the 2001–2011 lookup.

```
# Label propagation over lookup links: every LSOA in a connected component
# of 2001 <-> 2011 links ends up with the component's smallest label.
a <- paste0("01:", lookup$LSOA01CD)
b <- paste0("11:", lookup$LSOA11CD)
label <- setNames(seq_along(unique(c(a, b))), unique(c(a, b)))
repeat {
  m   <- pmin(label[a], label[b])
  low <- tapply(c(m, m), c(a, b), min)
  new <- label
  new[names(low)] <- pmin(label[names(low)], low)
  if (all(new == label)) break
  label <- new
}
lookup$component <- label[a]
lookup <- lookup %>%
  group_by(component) %>%
  mutate(unit_id = if (n_distinct(LSOA01CD) == 1) LSOA01CD
                   else paste0(min(LSOA01CD), "+", n_distinct(LSOA01CD) - 1),
         change  = case_when(any(CHGIND %in% c("M", "X")) ~ "merged/redrawn",
                             any(CHGIND == "S") ~ "split",
                             TRUE ~ "unchanged")) %>%
  ungroup()

units01 <- lsoa01 %>%
  left_join(distinct(lookup, LSOA01CD, unit_id), by = "LSOA01CD") %>%
  group_by(unit_id) %>%
  summarise(do_union = TRUE, .groups = "drop")

lsoa11_to_unit <- distinct(lookup, LSOA11CD, unit_id)
stopifnot(!anyDuplicated(lsoa11_to_unit$LSOA11CD))  # each 2011 LSOA in 1 unit
nrow(units01)
```

```
#> [1] 1640
```

3.2 One sf object per period

```
poly_list <- lapply(setNames(years, years), function(yr) {
  d <- filter(deaths11, year == yr)
  if (yr <= 2009) {                # 2001 geography
    counts <- d %>%
      inner_join(lsoa11_to_unit, by = c("lsoa11cd" = "LSOA11CD")) %>%
      group_by(AREA_ID = unit_id) %>%
      summarise(deaths = sum(deaths), .groups = "drop")
    units01 %>% rename(AREA_ID = unit_id) %>%
      left_join(counts, by = "AREA_ID")
  } else {                         # 2011 geography
    lsoa11 %>% rename(AREA_ID = LSOA11CD) %>%
```

```
      left_join(select(d, AREA_ID = lsoa11cd, deaths), by = "AREA_ID")
  }
})

# Offsets: one layer per period, named to match poly_list.
offset_list <- lapply(setNames(names(pop_u5), names(pop_u5)),
                      function(nm) pop_u5[[nm]])
```

Note the use of `left_join` with **no** `replace_na(deaths, 0)`: a polygon without a matched count should stop the analysis, not quietly become a zero (Section 9.1).

4 Pre-fit checks

These checks are cheap, and each one targets a failure we actually encountered. Run them before every fit, and in particular on any code generated by an AI assistant.

```
for (p in names(poly_list)) {
  x <- poly_list[[p]]
  d <- filter(deaths11, year == as.integer(p))
  # 1. every polygon has a count (no NA, no silent zero-fill)
  stopifnot(!anyNA(x$deaths))
  # 2. no count was lost when matching codes to polygons
  stopifnot(sum(x$deaths) == sum(d$deaths))
  # 3. polygon IDs are unique within the period
  stopifnot(!anyDuplicated(x$AREA_ID))
  # 4. polygons and offsets share a CRS
  stopifnot(st_crs(x) == st_crs(crs(offset_list[[p]])))
}
# 5. periods line up between the two lists
stopifnot(identical(names(poly_list), names(offset_list)))

# 6. every polygon has population in the offset
pop_in_poly <- sapply(names(poly_list), function(p)
  min(extract(offset_list[[p]], vect(poly_list[[p]]), fun = sum,
              na.rm = TRUE, ID = FALSE)[[1]]))
stopifnot(all(pop_in_poly > 0))
```

4.1 The boundary change

```
export_png <- function(file, expr, width = 5, height = 4.4) {
  if (knitr::is_latex_output()) return(invisible())
  dir.create(dirname(file), showWarnings = FALSE)
  png(file, width = width, height = height, units = "in", res = 300)
  expr
  invisible(dev.off())
}

centre <- ext(383900 - 3500, 383900 + 3500, 398200 - 3000, 398200 + 3000)

plot_boundaries <- function() {
  bg <- openmap(vect(centre, crs = uk_grid), zoom = 13)
  par(mar = c(0, 0, 0, 0))
  map.new(centre)
```

```
  plot(bg, add = TRUE)
  plot(crop(vect(lsoa11), centre), border = "#D55E00", lwd = 1.3, add = TRUE)
  plot(crop(vect(units01), centre), border = "black", lwd = 1.8, add = TRUE)
  legend("bottomright", lwd = c(1.8, 1.3), col = c("black", "#D55E00"),
         bg = "white", cex = 0.8, inset = 0.02,
         legend = c("2008-09 (2001 LSOAs)", "2010-12 (2011 LSOAs)"))
}
```

```
plot_boundaries()
```

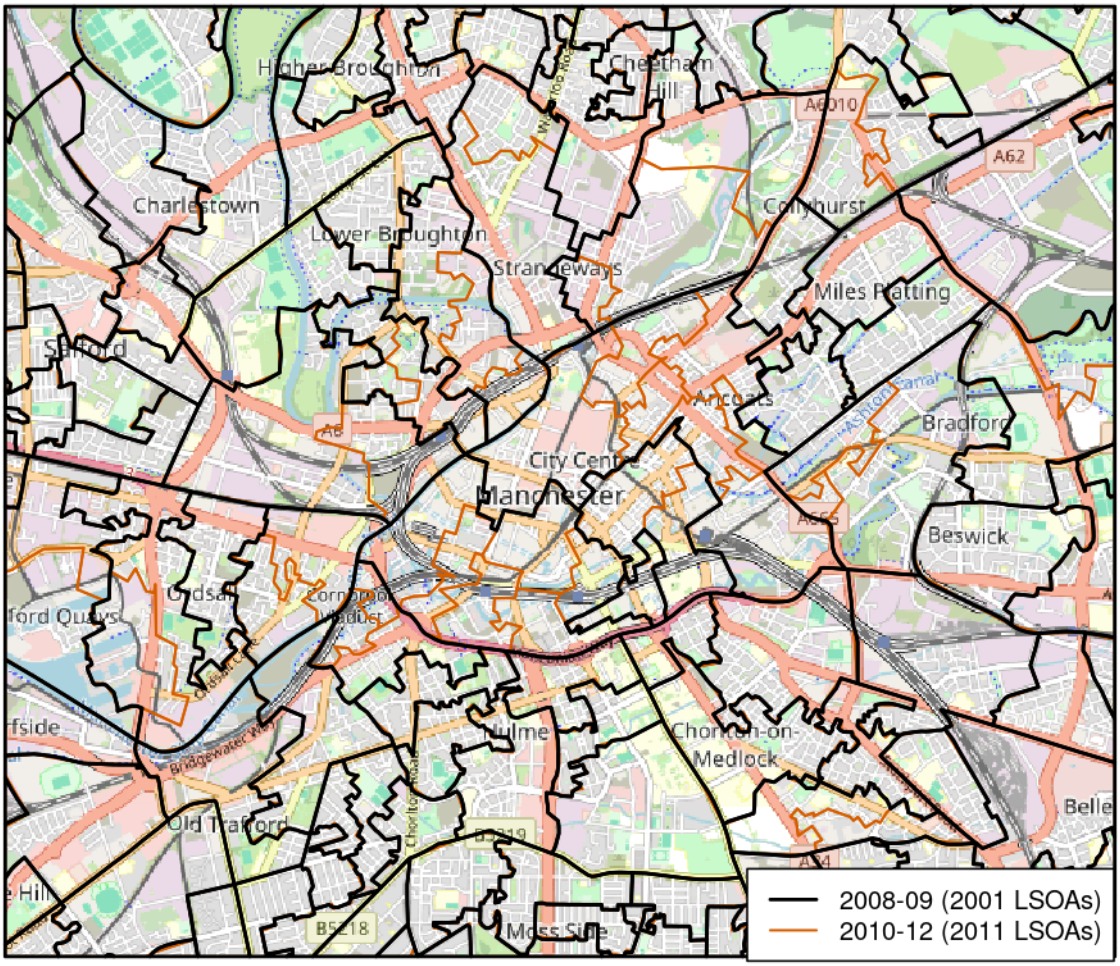


Figure 1: LSOA boundaries in Manchester city centre before and after the 2011 revision. Background map data © OpenStreetMap contributors.

```
export_png("output/manchester_boundaries.png", plot_boundaries(),
           width = 6, height = 5.2)
```

The figure shows the city centre, where the changes are concentrated: population growth during the 2000s led ONS to split 2001 LSOAs that had grown too large. Elsewhere in Greater Manchester the changes are scattered, and the two geographies mostly coincide.

5 Model specification

5.1 Mesh

The random field $u(s)$ is represented on a triangular mesh (the SPDE approach). `prepare_data_mmap()` builds the mesh from `mesh_args`, which are passed to `fmesher`:

- `max.edge = c(inner, outer)`: the largest triangle edge inside the study area and in the extension around it. The inner value should be well below the spatial range you expect to resolve.
- `cutoff`: points closer than this are merged, which stops dense polygon vertices from producing tiny triangles.
- `offset = c(inner, outer)`: the width of the extension around the study area, which keeps boundary effects away from the data.
- `resolution`: the size of the grid on which the mesh is evaluated for prediction.

We scale these by the diagonal of the study area, as in the Belgium–Netherlands example.

```
bb  <- st_bbox(poly_list[[1]])
diagonal <- sqrt((bb$xmax - bb$xmin)^2 + (bb$ymax - bb$ymin)^2)
maxedge  <- diagonal / 16

dis_data <- prepare_data_mmap(
  polygon_shapefile_list   = poly_list,
  aggregation_rasters_list = offset_list,
  response_var = "deaths",
  id_var       = "AREA_ID",
  na_action    = TRUE,
  mesh_args = list(max.edge = c(maxedge / 5, maxedge),
                   cutoff   = 0.2 * maxedge,
                   offset   = c(diagonal / 16, diagonal / 4),
                   resolution = 250)
)
c(diagonal_km = round(unname(diagonal) / 1000), mesh_nodes = dis_data$mesh$n)
```

```
#> diagonal_km  mesh_nodes
#>          67        1982
```

```
plot(vect(poly_list[["2010"]]), border = "grey60", lwd = 0.3)
points(dis_data$mesh$loc[, 1:2], pch = 15, cex = 0.25)
```

5.2 Priors

All variance and range parameters have penalised-complexity (PC) priors. Each prior is set by a statement about an implausible value rather than by choosing a density directly:

- **range** ρ: $\Pr(\rho < \rho_0) = p_\rho$, i.e. how likely a range shorter than ρ_0 metres is;
- **spatial SD** σ: $\Pr(\sigma > \sigma_0) = p_\sigma$;
- **overdispersion** τ: $\Pr(\tau > \tau_0) = p_\tau$. τ is the coefficient of variation of the gamma mixing distribution, so $\mathrm{Var}(Y) = \mu + \tau^2\mu^2$, and $\tau = 0$ is the Poisson model.

`get_priors()` returns the defaults computed from the data. We override three of them.

```
priors <- get_priors(dis_data)
defaults <- priors
priors$prior_rho_min           <- 2500  # Pr(range < 2.5 km) = 0.5
priors$prior_rho_prob          <- 0.5
priors$prior_iideffect_sd_max  <- 0.1   # Pr(tau > 0.1) = 0.5
priors$prior_iideffect_sd_prob <- 0.5
```

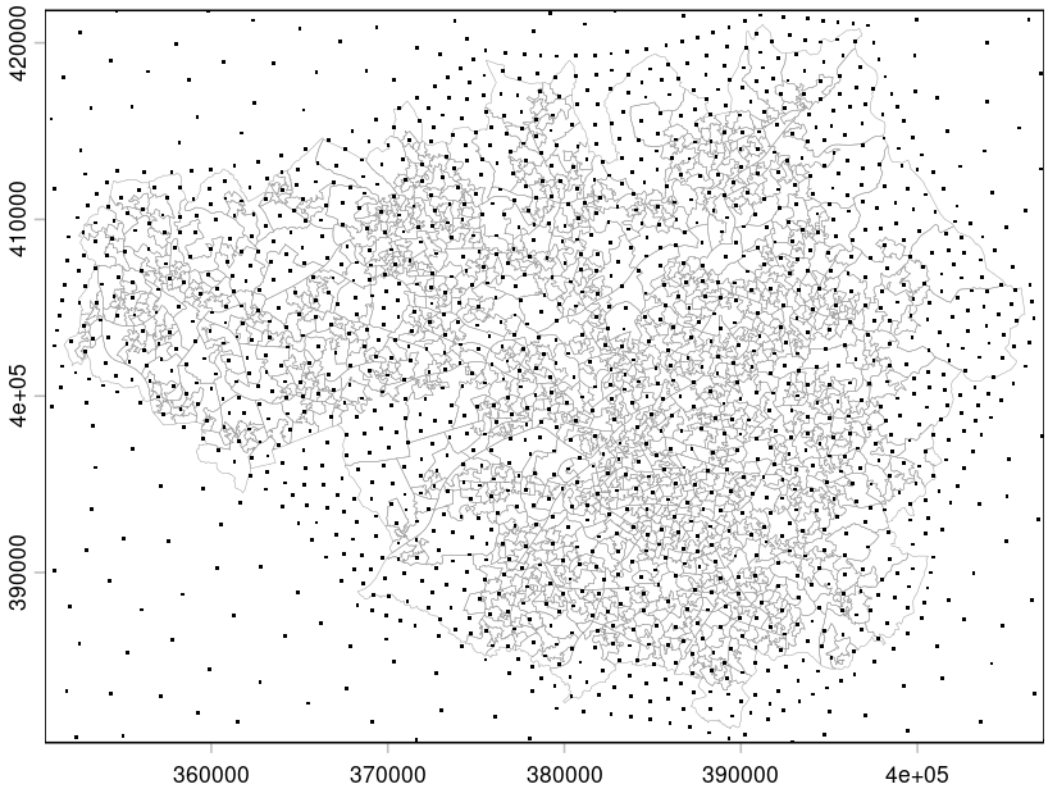


Figure 2: Mesh vertices over the 2010 LSOAs.

```r
priors$prior_sigma_max          <- 1      # Pr(sigma > 1) = 0.1

knitr::kable(data.frame(
  parameter = names(priors),
  default   = signif(unlist(defaults), 3),
  used      = signif(unlist(priors), 3)
), row.names = FALSE)
```

parameter	default	used
priormean_intercept	0.00	0.0
priorsd_intercept	10.00	10.0
priormean_slope	0.00	0.0
priorsd_slope	0.50	0.5
prior_rho_min	22500.00	2500.0
prior_rho_prob	0.10	0.5
prior_sigma_max	2.97	1.0
prior_sigma_prob	0.10	0.1
prior_iideffect_sd_max	0.10	0.1
prior_iideffect_sd_prob	0.01	0.5

The default range bound is a third of the study-area diagonal, which suits a smooth, regional surface. Here we want to resolve variation between neighbourhoods of a city, so we put half the prior mass on ranges shorter than 2.5 km. The default overdispersion prior, $\Pr(\tau > 0.1) = 0.01$, is strongly anchored at Poisson. We relax it to $\Pr(\tau > 0.1) = 0.5$ so that the data can show overdispersion if it is there. In either case the prior shrinks towards Poisson, so no separate Poisson-versus-negative-binomial test is needed: if the posterior for τ concentrates near zero, the fit is effectively Poisson (Section 7.3). The default bound on the spatial SD allows very large variation in risk. A spatial SD of 1 already lets the relative risk vary from about 0.14 to 7 (± 2 SD), so we use $\Pr(\sigma > 1) = 0.1$, as in the Belgium–Netherlands example.

5.3 Likelihood and intercepts

`family = "negbinomial"` uses the gamma-mixed (negative binomial) likelihood described in the paper. `family = "poisson"` with `iid = TRUE` gives the lognormal alternative instead. `time_varying_betas = TRUE` gives each year its own intercept β_i, which absorbs year-to-year changes in the overall rate. The shared field $u(s)$ is then a relative-risk surface common to all five years.

6 Fitting

```r
fit_time <- system.time(
  fit <- disag_model_mmap(
    data   = dis_data,
    family = "negbinomial",
    link   = "log",
    iid    = TRUE,
    priors = priors,
    engine = "AGHQ",
    engine.args = list(aghq_k = 2,
                       optimizer = "nlminb"),  # see Section 9.5
    time_varying_betas = TRUE
  )
```

```
)
round(fit_time[["elapsed"]] / 60, 1)   # minutes
```

```
#> [1] 54.1
```

`engine = "AGHQ"` integrates over the hyperparameters and fixed effects, $\theta = (\beta_{2008}, \ldots, \beta_{2012}, \log\tau, \log\sigma, \log\rho)$, with adaptive Gauss–Hermite quadrature. It uses `aghq_k` points per dimension, so k^8 points here, and integrates the latent field with a Laplace approximation at each point. `aghq_k = 1` is the Laplace approximation, which needs a single optimisation and is the fastest. `aghq_k = 2` (256 points) captures skewness in the posterior at a considerable cost. With `time_varying_betas = FALSE` there would be a single intercept, θ would have four dimensions, and `aghq_k = 2` would need 16 points. Develop with `aghq_k = 1`, and use `aghq_k = 2` for the final run. We use the `nlminb` optimiser for the outer optimisation because the default (BFGS) failed on these data (Section 9.5). On a desktop computer (one core in use) this fit took 54 minutes.

7 Outputs

7.1 Parameter summary

`summary(fit)` prints the full AGHQ summary, including the covariance matrix used for quadrature. Here we extract the parameter table, then put it on interpretable scales: the intercepts become baseline death rates per 1,000 children per year, and the range is given in kilometres.

```
invisible(capture.output(
  s <- summary(fit$aghq_model, max_print = 0)$summarytable
))
rownames(s) <- c(paste0("intercept_", years), "log_tau", "log_sigma",
                 "log_rho")
round(s, 3)
```

```
#>                   mean    sd   2.5% median  97.5%
#> intercept_2008 -6.936 0.101 -7.126 -6.932 -6.730
#> intercept_2009 -6.939 0.101 -7.128 -6.935 -6.735
#> intercept_2010 -6.999 0.102 -7.189 -6.995 -6.793
#> intercept_2011 -7.047 0.103 -7.239 -7.044 -6.840
#> intercept_2012 -7.120 0.104 -7.314 -7.116 -6.910
#> log_tau        -0.468 0.164 -0.774 -0.465 -0.132
#> log_sigma      -0.474 0.102 -0.672 -0.450 -0.277
#> log_rho         7.944 0.225  7.513  8.006  8.381
```

```
natural <- exp(s[, c("2.5%", "median", "97.5%")])
natural[1:5, ] <- 1000 * natural[1:5, ]          # deaths per 1,000 per year
natural["log_rho", ] <- natural["log_rho", ] / 1000   # metres to km
rownames(natural) <- c(paste0("rate_", years), "tau", "sigma", "range_km")
round(natural, 2)
```

```
#>           2.5% median 97.5%
#> rate_2008 0.80   0.98  1.19
#> rate_2009 0.80   0.97  1.19
#> rate_2010 0.76   0.92  1.12
#> rate_2011 0.72   0.87  1.07
#> rate_2012 0.67   0.81  1.00
#> tau       0.46   0.63  0.88
#> sigma     0.51   0.64  0.76
#> range_km  1.83   3.00  4.36
```

The baseline rate falls over the five years. The baseline is the rate where $u(s) = 0$, so it is somewhat lower than the crude rate (deaths divided by population), which is pulled up by the high-risk areas.

7.2 Relative-risk and exceedance maps

We draw 500 samples from the posterior of the latent field and project them onto the prediction grid. For each grid cell we compute the posterior median of $\exp\{u(s)\}$, the relative risk compared with the year-specific baseline, and the exceedance probability $\Pr\{u(s) > 0 \mid y\}$, the probability that risk at s is above that baseline.

```
set.seed(2026)
samples <- aghq::sample_marginal(fit$aghq_model, M = 500)
pm <- DAST:::get_predict_matrices(       # internal helper
  data = fit$data,
  expected_cov_names = fit$model_setup$coef_meta$cov_names,
  time_varying_betas = TRUE
)
field <- pm$A %*% samples$samps          # cells x samples, log scale
surface <- rast(
  data.frame(pm$coords,
             rr  = exp(apply(field, 1, median)),
             exc = apply(field, 1, function(z) mean(z > 0))),
  crs = uk_grid
)
surface <- mask(surface, vect(outline))
```

```
lad_outline <- lsoa11 %>%
  left_join(distinct(lookup, LSOA11CD, LAD11CD), by = "LSOA11CD") %>%
  group_by(LAD11CD) %>% summarise(do_union = TRUE, .groups = "drop") %>%
  vect()
rr_col  <- colourScale(surface[["rr"]], breaks = 9, style = "equal",
                       transform = "log", col = "Spectral", rev = TRUE,
                       dec = 2)
exc_col <- colourScale(breaks = c(0, 0.2, 0.5, 0.8, 0.95, 1),
                       style = "fixed", col = "RdYlGn", rev = TRUE)

# One map per figure: terra's add = TRUE does not mix with multi-panel
# layouts, so each map gets its own device.
plot_layer <- function(layer, col, title) {
  par(mar = c(0, 0, 0, 0))
  map.new(surface)
  plot(surface[[layer]], col = col$col, breaks = col$breaks,
       legend = FALSE, add = TRUE)
  plot(lad_outline, border = "grey20", lwd = 0.6, add = TRUE)
  legendBreaks("bottomleft", col, bty = "n", title = title, cex = 0.8)
}
```

```
plot_layer("rr", rr_col, "RR")
plot_layer("exc", exc_col, "Pr(RR>1)")
export_png("output/manchester_rr.png", plot_layer("rr", rr_col, "RR"))
export_png("output/manchester_exc.png",
           plot_layer("exc", exc_col, "Pr(RR>1)"))
```

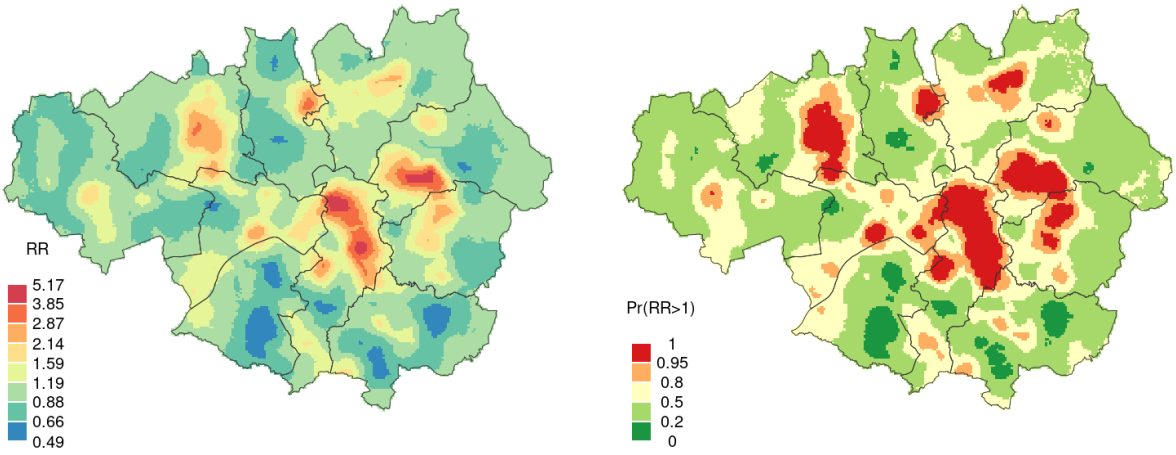


Figure 3: Under-5 mortality in Greater Manchester, 2008–2012. Left: posterior median relative risk. Right: posterior probability that relative risk exceeds 1. Lines are borough boundaries.

7.3 Prior and posterior distributions

Comparing prior and posterior shows how much the data inform each hyperparameter. A posterior that looks like the prior means the data had little to say, and the prior is driving the result.

```
# Gaussian approximation on the log scale at the posterior mode, transformed
# to the natural scale (as in the Belgium--Netherlands example).
mode <- fit$aghq_model$optresults$mode
covm <- solve(fit$aghq_model$optresults$hessian)
pr   <- fit$obj$env$data
prior_dens <- list(
  log_rho = function(x) {
    lam <- -log(pr$prior_rho_prob) * pr$prior_rho_min
    lam / x^2 * exp(-lam / x)
  },
  log_sigma = function(x)
    dexp(x, -log(pr$prior_sigma_prob) / pr$prior_sigma_max),
  iideffect_log_tau = function(x)
    dexp(x, -log(pr$prior_iideffect_sd_prob) / pr$prior_iideffect_sd_max)
)
labels <- c(log_rho = "range (km)", log_sigma = "spatial SD",
            iideffect_log_tau = "NB overdispersion tau")
par(mfrow = c(1, 3), mar = c(4, 4, 1, 1))
for (nm in names(prior_dens)) {
  i <- which(names(mode) == nm)
  z <- seq(mode[i] - 4 * sqrt(covm[i, i]), mode[i] + 4 * sqrt(covm[i, i]),
           length.out = 400)
  x <- exp(z)
  post  <- dnorm(z, mode[i], sqrt(covm[i, i])) / x   # Jacobian of exp()
  prior <- prior_dens[[nm]](x)
  scale <- if (nm == "log_rho") 1000 else 1
  matplot(x / scale, cbind(prior, post) * scale, type = "l", lty = c(2, 1),
          col = "black", xlab = labels[nm], ylab = "density", bty = "l")
```

```
}
```

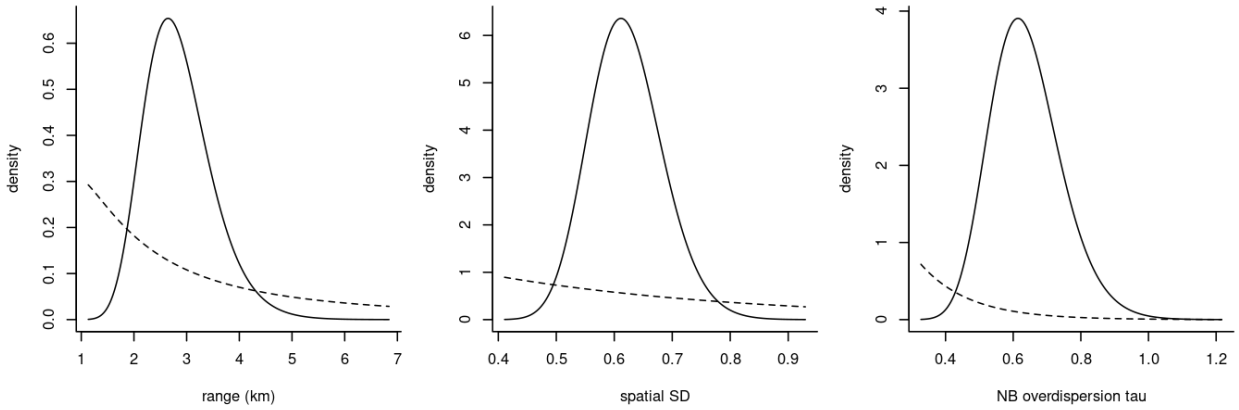


Figure 4: Priors (dashed) and approximate posteriors (solid) for the hyperparameters.

All three posteriors are much narrower than their priors, so the data are informative about these parameters. The posterior mode of the range is about 2.8 km, which is short compared with the conurbation and consistent with variation between neighbourhoods. The posterior for τ is centred near 0.63, well away from zero. The counts are therefore clearly overdispersed compared with Poisson, and the negative binomial likelihood matters here. Had the posterior for τ piled up at zero, the fit would have been effectively Poisson.

8 Interpretation

The posterior median relative risk ranges from 0.49 to 5.17 across the conurbation, with the central half of grid cells between 0.87 and 1.22. On its own, the median map invites over-reading: any smooth surface fitted to noisy counts will have highs and lows.

The exceedance map shows which of those highs and lows the data support. Only 8% of the area has $\Pr(\text{RR} > 1) > 0.95$, and less than 1% has $\Pr(\text{RR} > 1) < 0.05$. Over the remaining 92% the data do not distinguish risk from the baseline. That is the expected result with about 220 deaths a year spread over 1,650 areas, and it is the main reason to report exceedance alongside the median. The asymmetry is also informative. A few clusters of elevated risk are well supported, but almost nowhere is risk confidently *below* the baseline: with so few deaths, a further reduction from an already low rate is very hard to detect. The supported clusters can be followed up, while the rest of the map should not be interpreted area by area.

Two further cautions apply. First, without covariates the surface below LSOA scale is a smoothed interpolation, not new information about where deaths occur within an LSOA. Second, $u(s)$ is shared across years, so the maps summarise 2008–2012 as a whole and cannot show change over time.

9 Common issues and debugging

9.1 LSOA vintages and lookups

Symptom: counts silently disappear, or the fitted surface has holes or spurious lows in particular neighbourhoods. *Cause:* joining counts coded to one LSOA vintage onto polygons of another. Most codes are identical in 2001 and 2011, so a join "mostly works". The LSOAs that changed fail to match, and `replace_na(0)` turns them into areas with zero deaths. *Fix:* join through the official lookup and check totals before and after every join (check 2 in Section 4). For example, the join used in the AI-generated version of this analysis loses the following under-5 deaths:

```r
deaths11 %>%
  mutate(matched = lsoa11cd %in% lookup$LSOA01CD) %>%
  group_by(year) %>%
  summarise(total = sum(deaths), lost = sum(deaths[!matched]),
            .groups = "drop")
```

```
#> # A tibble: 5 x 3
#>    year total  lost
#>   <int> <dbl> <dbl>
#> 1  2008   221    11
#> 2  2009   228    11
#> 3  2010   220     6
#> 4  2011   213    12
#> 5  2012   202     5
```

9.2 Agencies re-code old years to new boundaries

Symptom: the data contain no boundary change where you expect one, or a newer download disagrees with an older one. *Cause:* agencies re-publish historical counts on current geographies. ONS supplies 2001–2019 deaths on 2011 LSOAs (Section 3.1). Eurostat has republished Belgian NUTS-3 counts from 2017 onwards on NUTS 2021 codes, which is why the Belgium–Netherlands example (`disagg_EU.Rmd`) uses NUTS 2016 boundaries for 2015–16, NUTS 2021 for 2017–22 and NUTS 2024 for 2023. *Fix:* for each period, confirm which boundary vintage the codes belong to, for example by checking which code set they match. Freeze the data with an extraction date (Section 2), and record that date in the paper.

9.3 Raster CRS and alignment

Symptom: offsets of zero or NA for some polygons, population totals that change after reprojection, or an error from `prepare_data_mmap()` about extents. *Cause:* WorldPop rasters are in longitude/latitude while the polygons are in metres. Reprojecting a count raster with bilinear interpolation does not preserve totals. *Fix:* reproject to the polygons' CRS with `method = "sum"` onto an explicit template grid. Check that each polygon has positive population (check 6) and compare totals before and after reprojection.

9.4 s2 versus GEOS topology errors

Symptom: `Loop 0 is not valid: Edge ... crosses edge ...` or `duplicate vertex` errors from `sf` when unioning or intersecting UK boundaries. *Cause:* since version 1.0, `sf` uses the spherical s2 engine for longitude/latitude data, and s2 is stricter than GEOS about polygon validity. *Fix:* `sf::sf_use_s2(FALSE)` at the start, transform to a projected CRS early, and apply `st_make_valid()` after reading.

9.5 AGHQ fails with `eigen()`: "infinite or missing values in 'x'"

Symptom: `disag_model_mmap()` stops after a long run with this error, raised from `mvQuad::rescale()` inside `aghq`. *Cause:* the Hessian of the hyperparameter posterior at its mode is not finite, so the quadrature grid cannot be built. The Hessian is computed by finite differences of the TMB gradient, and each evaluation needs an inner Laplace optimisation that can fail at some perturbed parameter values. This can happen with the default BFGS outer optimiser. *Fix:* switch the outer optimiser with `engine.args = list(aghq_k = 2, optimizer = "nlminb")`, which converged here. If that also fails, fit with `aghq_k = 1` to find the problem more quickly. Then look for polygons with tiny populations or for priors that allow implausible ranges, and consider fewer intercepts (`time_varying_betas = FALSE`).

9.6 Other checks worth running

- a map of the raw counts for each period, to catch polygons outside the study area or misplaced IDs;

- fit with `aghq_k = 1` first: if that fails, more quadrature points will not help.

10 Applying the method to your own data

The minimal inputs are:

1. **Polygons**, one `sf` object per period, in a projected CRS (metres), each with a unique ID column and a non-negative integer count column. The periods can have different polygons. Polygons should cover the same overall study region.
2. **Offsets**, one single-layer `SpatRaster` per period, holding the population at risk (counts, not densities) on a grid finer than the polygons, in the same CRS.
3. Both lists named by period, with the names in the same order.
4. Optionally, **covariates**, one `SpatRaster` per period with the same layer names in every period (`covariate_rasters_list`).

```
library(DAST)
library(sf)
library(terra)
sf::sf_use_s2(FALSE)

# 1. one sf per period: ID column + count column, projected CRS
poly_list <- list(
  "2015" = st_read("areas_2015.gpkg") |> st_transform(3035),
  "2016" = st_read("areas_2016.gpkg") |> st_transform(3035)
)
# 2. one population raster per period, same CRS, names match poly_list
offset_list <- list(
  "2015" = project(rast("pop_2015.tif"), "EPSG:3035", method = "sum"),
  "2016" = project(rast("pop_2016.tif"), "EPSG:3035", method = "sum")
)
# 3. checks from Section 4: no NA counts, totals preserved, unique IDs,
#    same CRS, names match, positive population in every polygon

# 4. data object and mesh (scale mesh_args to your study area)
dis_data <- prepare_data_mmap(
  polygon_shapefile_list = poly_list, aggregation_rasters_list = offset_list,
  response_var = "count", id_var = "area_id", na_action = TRUE)

# 5. priors: start from the defaults and change only what you can justify
priors <- get_priors(dis_data)

# 6. fit; try aghq_k = 1 first
fit <- disag_model_mmap(dis_data, priors = priors, family = "negbinomial",
                        engine = "AGHQ",
                        engine.args = list(aghq_k = 1, optimizer = "nlminb"),
                        time_varying_betas = TRUE)
summary(fit)
```

11 Using an AI assistant

The first version of this analysis was generated by an AI coding assistant (Codex), from a short prompt that pointed it at the Belgium–Netherlands example. That code is kept unchanged in `final_manchester_2008_2012.Rmd`.

What worked. The assistant found the right data sources, wrote working download code for them (reused here in `R/download_helpers.R`), and adapted the model-fitting code from the example correctly. That code is short and follows a fixed pattern.

What needed human checking. All the problems were in data assembly, and none of them produced an error:

- *study area*: the code selected all ten Greater Manchester boroughs when "Manchester" was asked for;
- *age range*: it summed deaths at ages 0–19, while the population offset was under-5;
- *boundary vintage*: it assumed the 2008–09 counts were on 2001 LSOAs. They are on 2011 LSOAs, so changed areas failed to match and were set to zero (the table in Section 9.1).

Our own debugging fixed the problems that caused errors. The boundary error is caught by checks 1 and 2 in Section 4. The study area and age range are not errors that a check of internal consistency can catch: they are found by reading the code against the question and by comparing the totals with a published figure.

Recommendations. Give the assistant a working example to adapt, as we did. This document is designed to be a very clear and strong example.

```
sessionInfo()
```

```
#> R version 4.6.1 (2026-06-24)
#> Platform: x86_64-pc-linux-gnu
#> Running under: Ubuntu 22.04.5 LTS
#>
#> Matrix products: default
#> BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.10.0
#> LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.10.0  LAPACK version 3.10.0
#>
#> locale:
#>  [1] LC_CTYPE=en_CA.UTF-8       LC_NUMERIC=C
#>  [3] LC_TIME=en_CA.UTF-8        LC_COLLATE=en_CA.UTF-8
#>  [5] LC_MONETARY=en_CA.UTF-8    LC_MESSAGES=en_CA.UTF-8
#>  [7] LC_PAPER=en_CA.UTF-8       LC_NAME=C
#>  [9] LC_ADDRESS=C               LC_TELEPHONE=C
#> [11] LC_MEASUREMENT=en_CA.UTF-8 LC_IDENTIFICATION=C
#>
#> time zone: America/Toronto
#> tzcode source: system (glibc)
#>
#> attached base packages:
#> [1] stats     graphics  grDevices datasets  utils     methods   base
#>
#> other attached packages:
#> [1] ggplot2_4.0.3 aghq_0.4.1    DAST_0.1.0    mapmisc_2.1.3 terra_1.9-27
#> [6] sf_1.1-3      dplyr_1.2.1
#>
#> loaded via a namespace (and not attached):
#>  [1] gtable_0.3.6         TMB_1.9.25           xfun_0.62
#>  [4] bslib_0.12.0         lattice_0.23-1       bspm_0.5.7
#>  [7] numDeriv_2016.8-1.1  vctrs_0.7.3          tools_4.6.1
#> [10] generics_0.1.4       tibble_3.3.1         proxy_0.4-29
#> [13] pkgconfig_2.0.3      Matrix_1.7-6         KernSmooth_2.23-27
#> [16] data.table_1.18.6.1  RColorBrewer_1.1-3   S7_0.2.2
#> [19] lifecycle_1.0.5      compiler_4.6.1       farver_2.1.2
#> [22] fmesher_0.8.0        statmod_1.5.2        codetools_0.2-20
#> [25] mvQuad_1.0-10        htmltools_0.5.9      class_7.3-24
```

```
#> [28] sass_0.4.10          yaml_2.3.12          pillar_1.11.1
#> [31] jquerylib_0.1.4      tidyr_1.3.2          classInt_0.4-11
#> [34] cachem_1.1.0         tidyselect_1.2.1     digest_0.6.39
#> [37] purrr_1.2.2          fastmap_1.2.0        grid_4.6.1
#> [40] cli_3.6.6            magrittr_2.0.5       broom_1.0.13
#> [43] e1071_1.7-17         withr_3.0.3          scales_1.4.0
#> [46] backports_1.5.1      sp_2.2-4             rmarkdown_2.32
#> [49] otel_0.2.0           evaluate_1.0.5       knitr_1.52
#> [52] splancs_2.01-45      disaggregation_0.4.1 rlang_1.3.0
#> [55] Rcpp_1.1.2           polynom_1.4-1        glue_1.8.1
#> [58] DBI_1.3.0            rSPDE_2.6.0          jsonlite_2.0.0
#> [61] R6_2.6.1             units_1.0-1
```